

POSTECH



TECH CHALLENGE

Tech Challenge é o projeto da fase que englobará os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, a princípio, deve ser desenvolvida em grupo. É importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale 90% da nota de todas as disciplinas da fase.

- Atividade em grupo · Obrigatória · Avaliada.
- Entrega obrigatória: Repositório GitHub + Vídeo de 5 minutos (método STAR).
- Entrega opcional: Deploy em ambiente de produção em nuvem.

O problema

Uma empresa de e-commerce precisa de um sistema preditivo para identificar a **propensão de compra** de um usuário baseado em seu comportamento de navegação.

O foco deste desafio não é a complexidade matemática do modelo, mas sim a **Engenharia de Machine Learning**. O grupo deverá treinar um modelo clássico de classificação (ex.: Scikit-Learn), com foco em construir um pipeline completo containerizado em Docker, dados versionados com DVC, experimentos rastreados no MLflow e código seguindo padrões profissionais de Clean Code.

Requisitos Obrigatórios

Repositório GitHub

- **Estrutura Clean Code:** módulos curtos, nomes descritivos, princípios de POO ou Funcional e uso de type hints.
- **Gerenciamento:** pyproject.toml com Poetry (ou uv), dependências prod/dev separadas, lock file commitado.
- **Configuração:** .dockerignore, .gitignore, e .env.example devidamente configurados.

- Histórico de commits organizado que evidencie o trabalho em grupo.

Vídeo (5 minutos — método STAR)

- **Situation:** problema de negócio e contexto do dataset.
- **Task:** objetivos técnicos e restrições.
- **Action:** decisões de arquitetura de código, estruturação do Docker, uso do DVC e MLflow.
- **Result:** resultados obtidos, o pipeline funcionando de ponta a ponta e lições aprendidas.

Bibliotecas Requeridas

- **Scikit-Learn** — para pré-processamento e treinamento do modelo de classificação.
- **MLflow** — tracking de experimentos e Model Registry.
- **DVC** — versionamento de dados e pipeline reproduzível.

Boas Práticas Obrigatórias

- **Clean Code:** funções curtas (idealmente ≤ 20 linhas), naming conventions e type hints.
- Uso adequado de Programação Orientada a Objetos (POO) ou Funcional para estruturar o pipeline.
- Dockerfile funcional e bem estruturado.
- Pipeline básico configurado no DVC (dvc.yaml).
- Seeds fixados para reprodutibilidade.

Etapas de Desenvolvimento (4 Etapas)

Etapa 1 — Clean Code e Estrutura (Disciplina 01)

Foco: Projeto limpo com padrões de engenharia desde o início.

- Tarefas:
 - Definir estrutura de projeto com pastas organizadas (ex.: src/, tests/, data/, models/, configs/).

- Aplicar naming conventions e princípios básicos de Clean Code.
- Utilizar type hints nas funções principais e adicionar docstrings.
- Opcional recomendado: Configurar linting (ex.: Ruff ou Flake8).
- **Entregável:** repositório base com estrutura limpa e código legível.

Etapas 2 — Ambiente e Dependências (Disciplina 02)

Foco: Reprodutibilidade garantida com gerenciamento moderno de dependências.

- Tarefas:
 - Configurar pyproject.toml utilizando Poetry. Separar dependências de produção (ex.: scikit-learn, mlflow) das de desenvolvimento (ex.: pytest, ruff).
 - Gerar e commitar o lock file (poetry.lock).
 - Externalizar variáveis e configurações para um arquivo .env (disponibilizando apenas o .env.example no repositório).
- **Entregável:** projeto instalável do zero em qualquer máquina com o comando poetry install.

Etapas 3 — Containerização e Versionamento (Disciplinas 03 e 04)

Foco: Docker e DVC integrados em um pipeline reprodutível.

- Tarefas:
 - Inicializar o DVC localmente para versionar o dataset escolhido.
 - Criar um pipeline via DVC (dvc.yaml) com estágios claros (ex.: preprocess → train).
 - Criar um Dockerfile que instale as dependências via Poetry e execute o projeto.
- **Entregável:** pipeline de dados versionado pelo DVC e aplicação capaz de rodar via Docker.

Etapla 4 — Modelagem, Registry e Entrega (Disciplina 04 + Consolidação)

Foco: Modelo de Machine Learning treinado, rastreado e documentado.

- Tarefas:
 - Treinar um modelo de classificação simples (ex.: Random Forest, Regressão Logística, XGBoost) usando Scikit-Learn.
 - Utilizar o **MLflow Tracking** para logar os parâmetros, métricas e o modelo final de cada execução (run).
 - Registrar o melhor modelo utilizando o **MLflow Model Registry**.
 - Finalizar o README.md com as instruções completas de como rodar o projeto e o DVC.
 - Gravar o vídeo STAR de 5 minutos demonstrando o pipeline rodando.
- **Entregável:** repositório final validado, experimentos rastreados no MLflow e link do vídeo.

Critérios de Avaliação

Critério	Peso	Descrição
Clean code e Estrutura	20%	Naming, type hints, legibilidade, uso correto de POO/Funcional.
Reprodutibilidade	20%	Uso correto do Poetry, lock file presente, variáveis no .env, instalação limpa.

Docker	15%	Dockerfile configurado corretamente para o projeto.
DVC + Pipeline	15%	Dataset versionado e dvc.yaml operante.
Modelagem Clássica	10%	Modelo funcional no Scikit-Learn resolvendo o problema proposto.
MLflow + Registry	20%	Runs rastreados corretamente, métricas logadas e modelo promovido no Registry.

Dataset Sugerido

Recomendamos a escolha de um dataset focado em classificação binária, preferencialmente voltado ao comportamento do cliente, para manter a consistência da temática de e-commerce e propensão de compra.

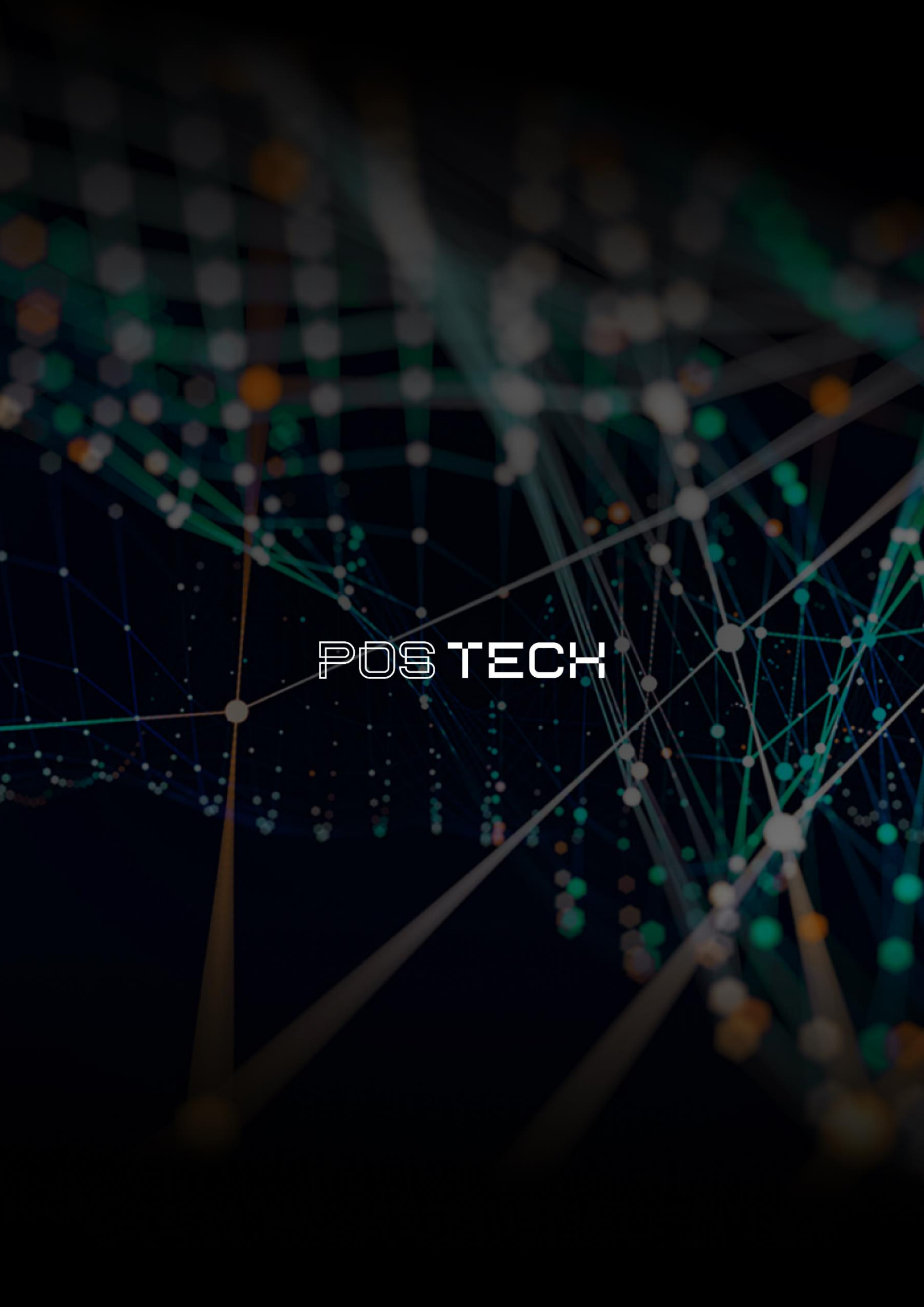
- **Exemplos:** Online Shoppers Purchasing Intention Dataset (UCI / Kaggle) ou dados tabulares de Customer Churn. Alternativa: qualquer dataset tabular de classificação binária com pelo menos 5.000 registros.

Passo a Passo Resumido

- **[Etapa 1]** Estruturação das pastas do repositório, Clean Code, type hints.
- **[Etapa 2]** Configuração do Poetry, geração do lock file, criação do .env.example.
- **[Etapa 3]** Criação do Dockerfile e versionamento do fluxo de dados com DVC.

- **[Etapa 4]** Treinamento do modelo Scikit-Learn rastreado pelo MLflow + Model Registry + Vídeo STAR.

Caso tenha qualquer dúvida, não deixe de nos procurar no Discord!

The background is a dark, abstract network visualization. It features a complex web of thin, light-colored lines connecting numerous small, glowing nodes. The nodes are primarily white and light blue, with some larger, more prominent nodes in teal and orange. The overall effect is a sense of dynamic connectivity and data flow, typical of a digital or technological theme.

POSTECH